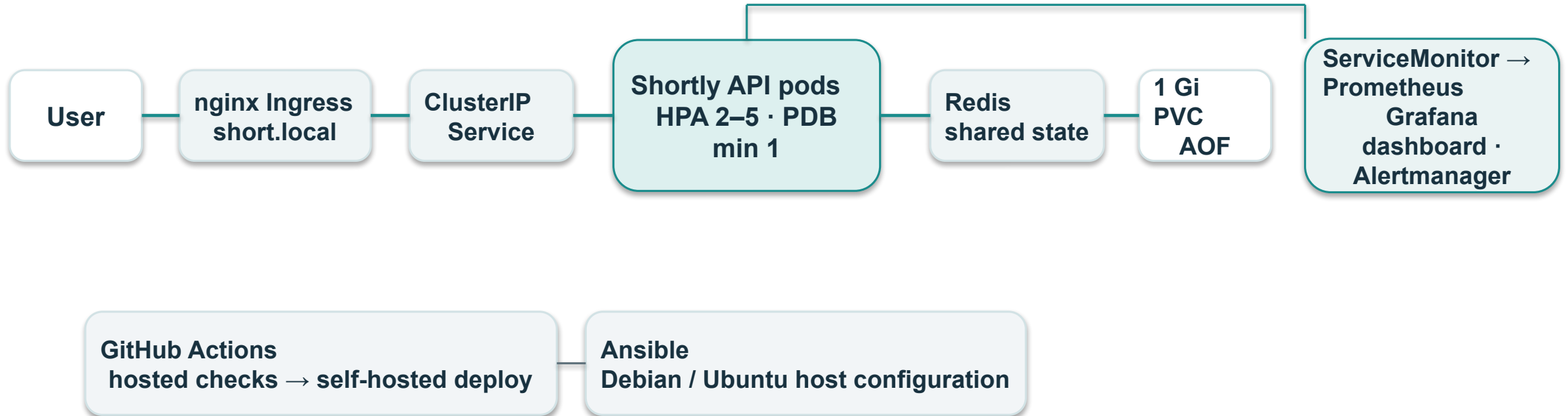


# Shortly keeps request state in Redis and releases under observation

Shortly · URL short links · FastAPI / Redis / Minikube / Prometheus / Locust / Ansible



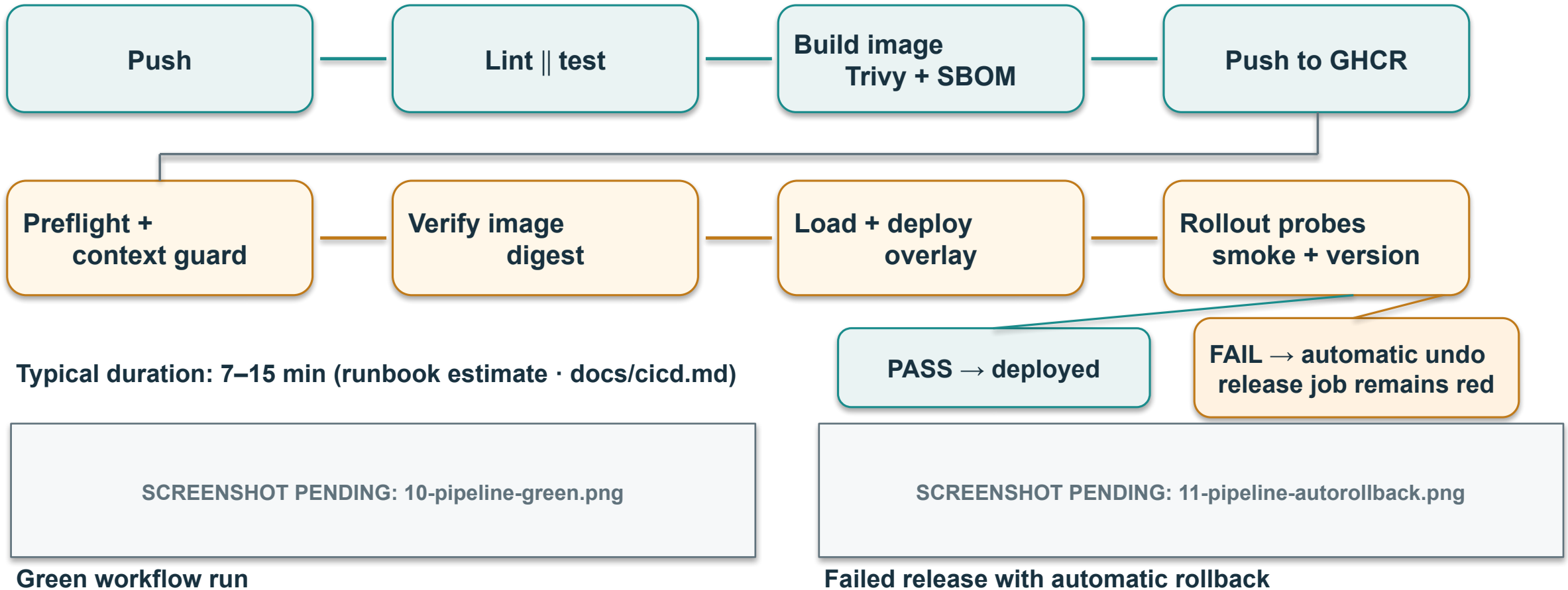
**Key choices: stateless API + Redis · readiness checks Redis; liveness does not**  
**maxUnavailable: 0 · one Redis replica in this local demo**

Metrics scrape the app Service; the local Alertmanager uses a null receiver.

# A scanned image must pass local-cluster gates before it deploys

Hosted checks protect builds; a guarded self-hosted runner updates Minikube.

## GITHUB-HOSTED RUNNERS



# Saved scenarios show resilience, with one suite caveat

Committed load-test and Ansible evidence; missing UI captures remain pending.

SCREENSHOT PENDING: 01-baseline-overview.png

**Grafana overview under baseline traffic**

SCREENSHOT PENDING: 17-rolling-update-probe.png

**Rolling update: v1 and v2 seen; 0 failures**

SCREENSHOT PENDING: 07-bad-release-crash.png

**CrashLoopBackOff observed, then rollback**

SCREENSHOT PENDING: 23-ansible-idempotent-recap.png

**Ansible second apply: changed=0**

**Saved run: 8/9 PASS, 1 FAIL; focused abuse rerun PASS**

**Baseline 3,753 requests / 0 failures · Surge 2→3 replicas · full suite 55.7 min**

The full suite was not rerun after the focused abuse fix. Source: loadtest/reports/SUMMARY.md.

# The hardest failures crossed component boundaries

Top evidence-backed challenges · details and sources: docs/CHALLENGES.md

**1 Ready probes missed release errors → health checks were too narrow → alert on 5xx and roll back**

2 Nested Kustomize base was blocked → path restriction → render a temporary sibling overlay

3 Grafana OOM at 256/384 MiB → limits were too low → use the evidenced 512 MiB setting

4 Redis outage broke readiness → API depends on Redis → separate liveness and verify recovery

5 Ansible laptop was macOS → playbook supports Debian/Ubuntu → prove idempotence on Ubuntu 24.04

# We learned to verify behavior, not only health

Each lesson links to the challenge log; production work is future work.

**#1 Readiness and user success are different signals; alert on real request outcomes.**

#2 Render the exact manifest set before applying; layout affects deploy tooling.

#3 Resource limits need observed headroom; lower Grafana limits restarted the pod.

#4 Dependency-aware readiness protects traffic without restarting the API process.

#5 Idempotence needs a supported target; keep the macOS refusal distinct.

**Next: use a multi-node cloud cluster; add SLO alerts, real alert routing and GitOps; isolate the self-hosted runner.**